

Desafío #1. Informe preliminar

Implementación de Sweet Crush mediante manipulación de bits en C++

Oscar Mauricio Echeverri Parra.

Luna Jaramillo Guzmán

Analisis del problema

Para este desafío debemos realizar una versión del juego Sweet Crush en C++, teniendo como parte fundamental la manipulación de bits y el uso de memoria dinámica.

La idea que entendimos inicialmente es que el tablero no se puede guardar utilizando una variable completa para cada posición. Cada ficha debe representarse utilizando únicamente 3 bits.

Por esta razón, primero debemos encontrar una forma de representar las fichas y después una manera de ubicarlas dentro de una secuencia continua de bits.

También debemos tener en cuenta que una ficha puede quedar repartida entre dos bytes, por lo que el programa debe poder encontrarla y modificarla correctamente.

El tablero tendrá unas filas y columnas determinadas, pero estas también podrán cambiar durante el funcionamiento del programa.

Representación de las fichas

Es necesario emplear 3 bits para representar cada ubicación en el tablero.

Con 3 bits se pueden generar 8 combinaciones posibles. De estas, asignaremos distintas combinaciones para representar las fichas y reservaremos algunas para otros estados.

bits	Representación
000	vacío
001	A
010	B
011	C
100	D
101	E
110	F
111	Reservado

Las fichas serán representadas mediante las letras A, B, C, D, E y F. La posición vacía se representará con V y la posición reservada con R.

La razón para emplear esta representación es que permite almacenar cada posición utilizando únicamente 3 bits. Esta representación puede ajustarse a lo largo del desarrollo si encontramos una manera más eficiente, pero siempre conservando la restricción de usar 3 bits por posición.

Organización de los bits y memoria

El tablero se almacenará como una secuencia continua de bits. Por ejemplo, si asignamos:

A = 001

B = 010.

C = 011

la secuencia resultante sería:

001010011

No se añadirán bits extras entre las fichas. Dado que un byte tiene 8 bits y cada posición utiliza 3 bits, algunas fichas pueden encajar completamente dentro de un byte, mientras que otras pueden quedar separadas entre dos bytes.

Para determinar la cantidad de memoria requerida, primero calculamos:

Número de posiciones = filas $\times$ columnas

Luego:

Bits necesarios = posiciones $\times$ 3

Finalmente, obtenemos el número de bytes requeridos, redondeando hacia arriba.

forma de acceso a una ficha

Para localizar una posición en el tablero, utilizaremos la posición lineal de la ficha. Primero determinamos dónde inician sus bits:

bitInicio = posición $\times$ 3

Luego calculamos:

númeroByte = bitInicio / 8

y también:

desplazamiento = bitInicio % 8

De esta forma, podemos identificar en qué byte se encuentra la ficha y desde qué bit debemos comenzar a leer. Para extraer y modificar los bits, utilizaremos operadores como:

>> para desplazar bits hacia la derecha,

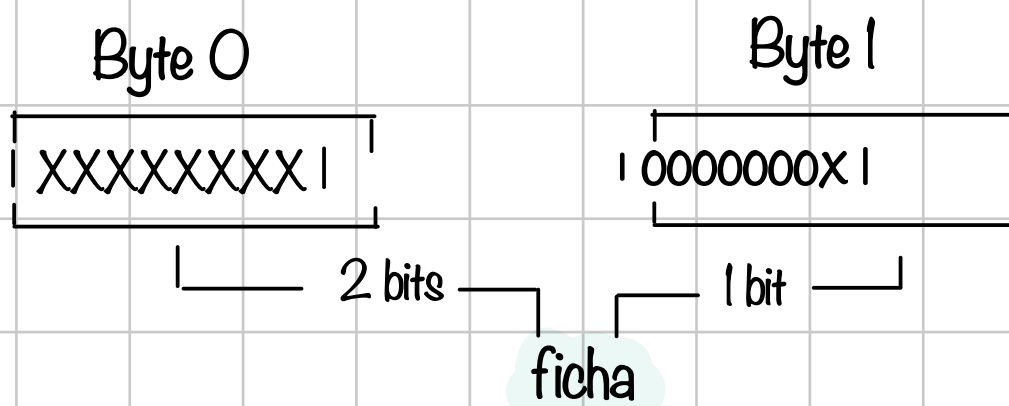
<< para desplazarlos hacia la izquierda,

& para obtener únicamente los bits necesarios,

y | para combinar o establecer bits..

¿ficha dividida en 2 bytes?

Uno de los inconvenientes que identificamos antes de iniciar la implementación es que una ficha de 3 bits no siempre se puede almacenar completamente en un solo byte.



En este caso, debemos leer una porción de la ficha del primer byte y la parte restante del segundo byte. Por lo tanto, no podemos suponer que cada ficha encaja completamente en un solo byte. La solución que proponemos es determinar primero cuánto de la ficha se encuentra en el primer byte y cuánto en el segundo, para luego combinar ambas partes usando operaciones de bits.

Funcionamiento del juego

La idea general del funcionamiento sería:

1. Pedir las dimensiones iniciales del tablero.
2. Calcular la memoria necesaria.
3. Crear la memoria dinámicamente.
4. Llenar las posiciones con fichas.
5. Mostrar el tablero.
6. Permitir seleccionar una posición.
7. Eliminar la ficha seleccionada.
8. Revisar las combinaciones horizontales y verticales.
9. Eliminar las posiciones correspondientes.
10. Aplicar la gravedad.
11. Generar nuevas ficha.
12. Volver a revisar si aparecieron nuevas combinaciones.
13. Repetir el proceso hasta que no existan nuevas combinaciones.

INICIO



Crear tablero



Llenar con fichas



Mostrar tablero



Seleccionar posición



Eliminar ficha



Buscar combinaciones



¿Se encontraron?

— NO → continuar

— SÍ



Eliminar



Gravedad



Generar fichas



Buscar nuevamente



¿Hay otra combinación?



repetir

Algoritmo

La intención es que las combinaciones puedan ser de tres o más fichas consecutivas.

También debemos considerar que una misma posición puede participar en una combinación horizontal y en una vertical.

Cambios en fila y columna

Otro requisito crucial a tener en cuenta es que el tamaño del tablero puede cambiar. Debemos permitir la adición o eliminación de:

- filas
- columnas

Además, estas modificaciones pueden hacerse en posiciones intermedias del tablero.

ANTES	Si agregamos una fila en el medio	Al eliminar una columna
A B C D E F G H I	A B C X X X D E F G H I	A B C A C D E F → D F G H I G I

Después de realizar estos cambios, será necesario reorganizar las posiciones y actualizar las dimensiones del tablero. Dado que las fichas están almacenadas como una secuencia de bits, este proceso puede requerir que muchos bits cambien de posición.

Manejo de memoria

La memoria debe ser dinámica y adaptarse al tamaño actual del tablero. No planeamos crear desde el principio una memoria para un tablero máximo, sino calcular la cantidad necesaria según el número de filas y columnas que tenga en ese momento.

Además, debemos considerar la condición del 65% indicada en la guía. Nuestra propuesta es verificar la utilización de la memoria después de eliminar filas o columnas. Si la utilización queda por debajo del 65%, se reorganizará la memoria para ajustar el tamaño. Si no se cumple esta condición, conservaremos la memoria asignada.

Evolución que se espera

Representación de 3 bits



Cálculo de memoria



Lectura de una posición



Modificación de una posición



Creación del tablero



Mostrar fichas y bits



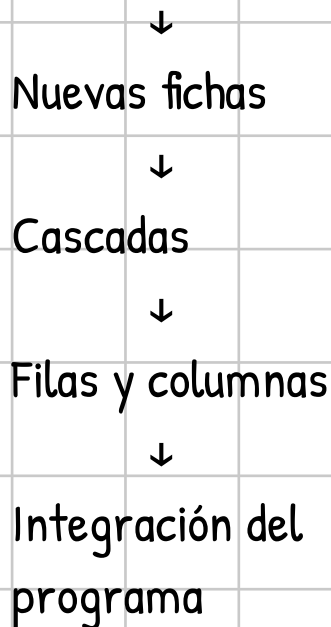
Detección de combinaciones



Eliminación



Gravedad



¿Cómo sería su implementación?

la implementación del juego se puede dividir en 2 partes principales

1. Representación del tablero

2. Lógica del juego

Parte 1 → Representación del tablero

- * Crear memoria dinámica

unsigned char* tablero

- * Funciones para trabajar con posiciones

Obtener ficha:

Calcular la posición de los 3 bits en el byte correspondiente.

Extraer los bits y devolver la ficha

Modificar ficha:

Limpiar los bits en la posición deseada.

Colocar los nuevos bits según la ficha a insertar.

Mostrar tablero:

Crear una función que recorra el puntero tablero y muestre el contenido de manera visual.

parte 2 -- lógica del juego

Recorrer el tablero:

Implementar dos bucles: uno para filas y otro para columnas.

Comparar fichas consecutivas.

Contar cuántas fichas iguales aparecen.

Identificar combinaciones:

Horizontal:

A B B B C → Marcar posiciones de las fichas iguales.

Vertical:

A

B

B

C → Marcar también estas posiciones

Marcar posiciones:

Crear una estructura para marcar las posiciones que deben ser eliminadas, evitando interferencias entre combinaciones.

Eliminar fichas:

Una vez marcadas las posiciones, reemplazar esas posiciones en el tablero con un valor que represente un espacio vacío.

Aplicar gravedad:

Reorganizar el tablero para que las fichas "caigan" hacia abajo, llenando los espacios vacíos.

Generar nuevas fichas:

Para cada espacio vacío, generar una nueva ficha aleatoriamente y colocarla en el tablero.